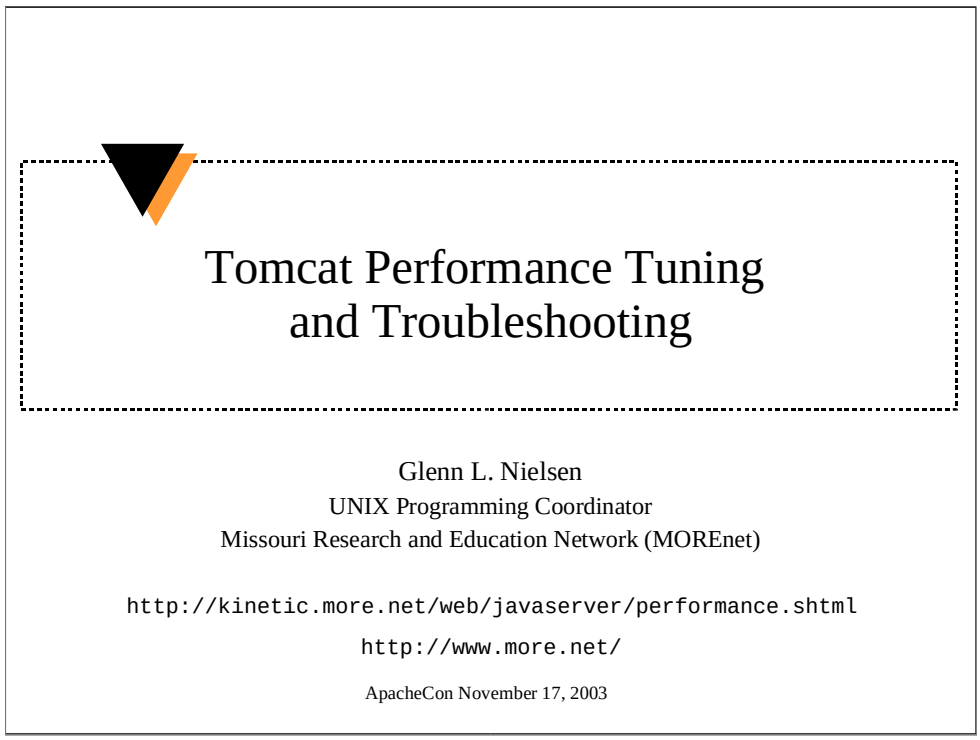
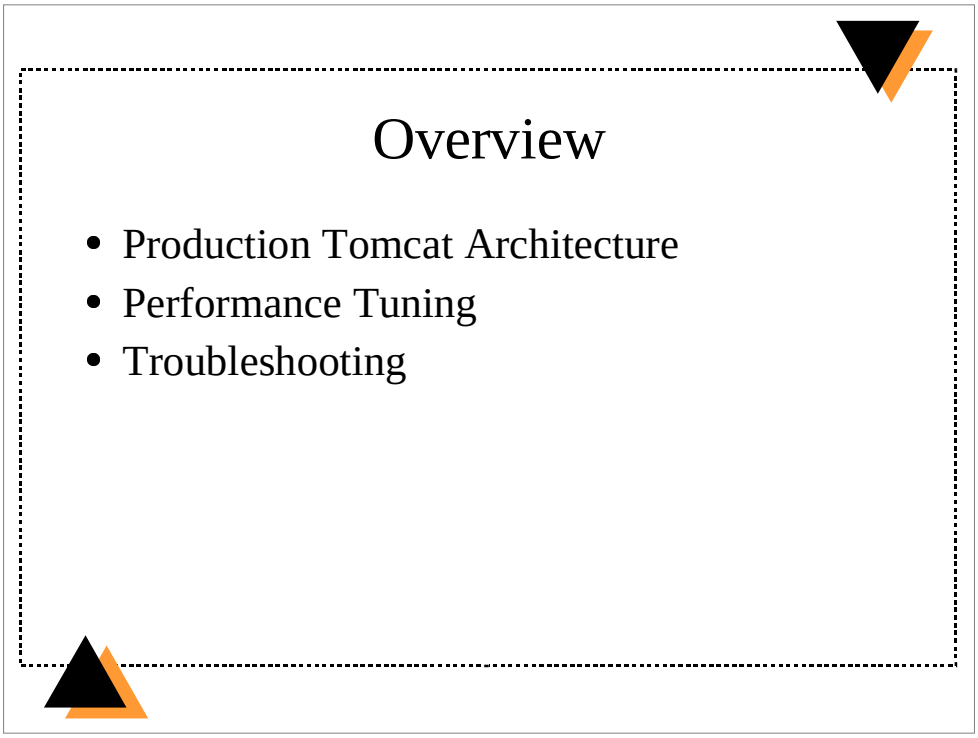


Tuning Tomcat for performance and resolving problems which affect availability are critical for a production instance of Tomcat. This session addresses how to collect and analyze data to trouble shoot problems and configure Tomcat for performance.



Notes

Production Tomcat Architecture

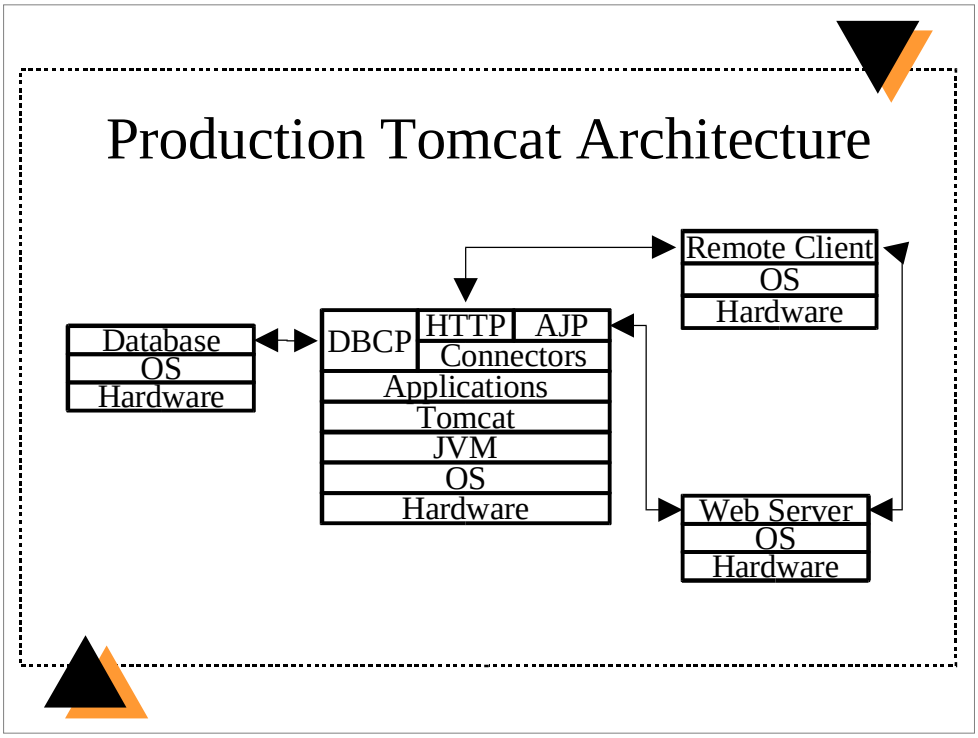
In production Tomcat relies on a number of resources which can all impact its overall performance. Understanding the overall system architecture is key to tuning performance and troubleshooting problems.

Performance Tuning

- How to measure and test performance
- JVM version, memory usage, and garbage collection
- Tomcat version and configuration
- Database connection pools
- Application Design and Profiling

Troubleshooting

- Collecting and analyzing log data
- Common problems



Notes

Hardware. CPU(s), memory, network IO, and file IO

Operating System. Symmetric multiprocessing (SMP) and thread support

JVM. Version, tuning memory usage, and tuning garbage collection are important.

Tomcat. Later releases are more optimized for performance. If you use JavaServer Pages, Jasper 2 available in Tomcat 4.1 significantly boosts performance.

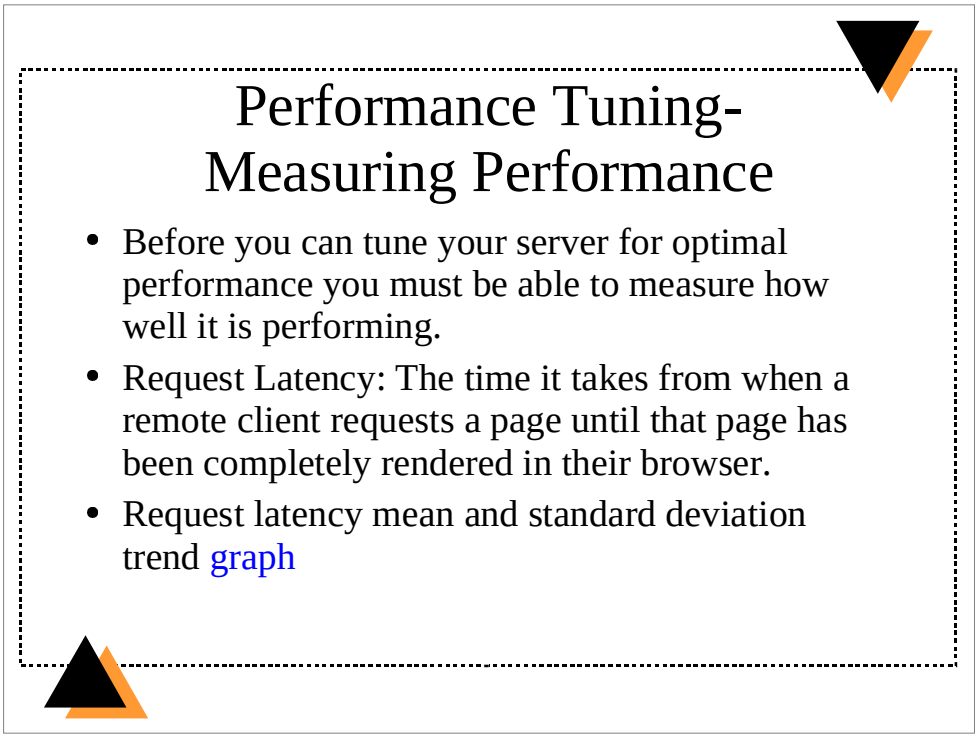
Application. Application design can have the largest impact on overall performance.

Database. Conncurrent db connections allowed, db connection pooling, and object caching.

Web Server. Can be used in front of Tomcat to serve static content removing this load from Tomcat. This allows Tomcat to do what it does best, dynamic web application content. Allows load balancing using multiple instances of Tomcat.

Network. Network delays. Nothing we can do about this.

Remote Client. Speed of remote client network connection. Nothing we can do about this except compress the content using Apache mod_gzip or Tomcat compression filter.



Notes

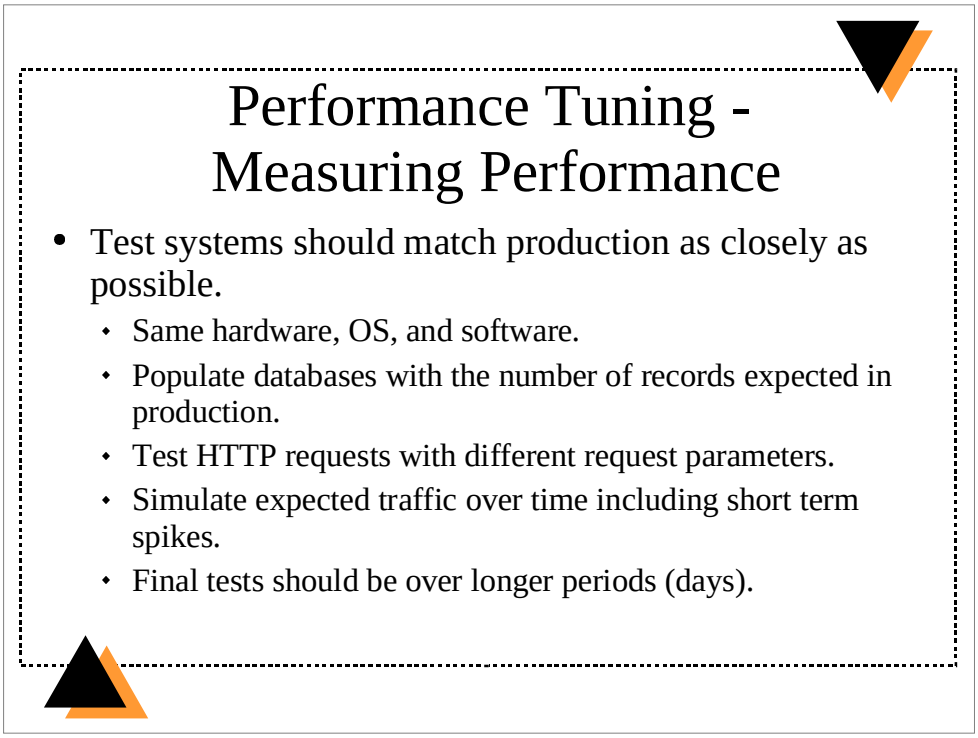
Without testing and measuring performance how can you know if any changes made have improved performance?

Request latency is the most important performance measurement because it reflects the responsiveness of your site for visitors.

You can only control the portion of the overall request latency which occurs while processing the request on your servers.

There is nothing you can do about the speed of the remote clients network connection or any network delays between your servers and the remote client.

http://kinetic.more.net/ar/kinetic/tomcat_deviation.png



Notes

Databases

An application may perform well with a small number of records in the database. Performance could degrade a great deal as the number of records in the table(s) increases.

Request parameters

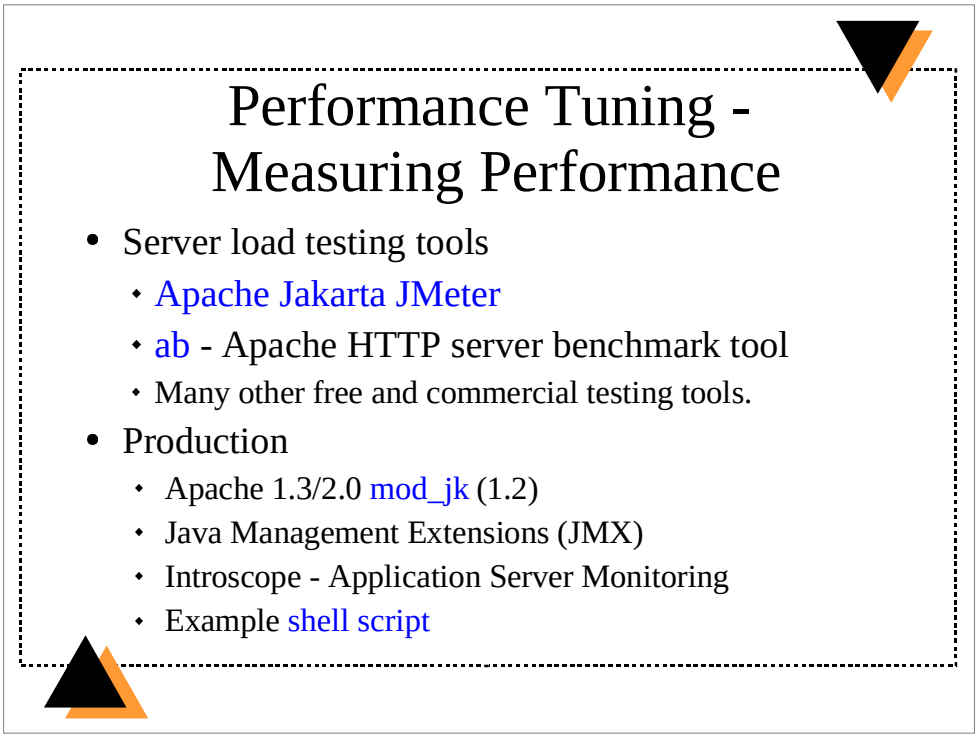
Test with the extremes. A page which performs a search may perform well when the search criteria returns a small result set, but perform poorly when the search criteria returns a large result set.

Simulated request traffic

A page which performs well with low request volume can cause the server to fail under higher request volume.

Longer tests

The JVM performance changes over time and can actually improve if using HotSpot. Memory leaks, db temporarily unavailable, etc. can only be found when running longer tests.



Notes

Apache Jakarta Jmeter

http://jakarta.apache.org/jmeter

Apache HTTP server benchmarking tool

http://httpd.apache.org/docs/programs/ab.html

mod_jk

If you are using Apache 1.3 and mod_jk 1.2 you can use JkRequestLogFormat to log individual Tomcat request latency in nanoseconds. The latest mod_jk 1.2 distribution comes with some perl scripts for analyzing mod_jk logs. The request latency graph we viewed in slide 4 was generated with these scripts.

http://jakarta.apache.org/tomcat/tomcat-4.1-doc/jk2/jk/aphowto.html

Java Management Extensions (JMX)

Tomcat currently uses JMX for configuration management. Using JMX to instrument Tomcat for production monitoring is being considered by Tomcat developers.

Open-source JMX API (MX4J) http://mx4j.sourceforge.net/

Open-source JMX Management Console (MC4J) http://mc4j.sourceforge.net/

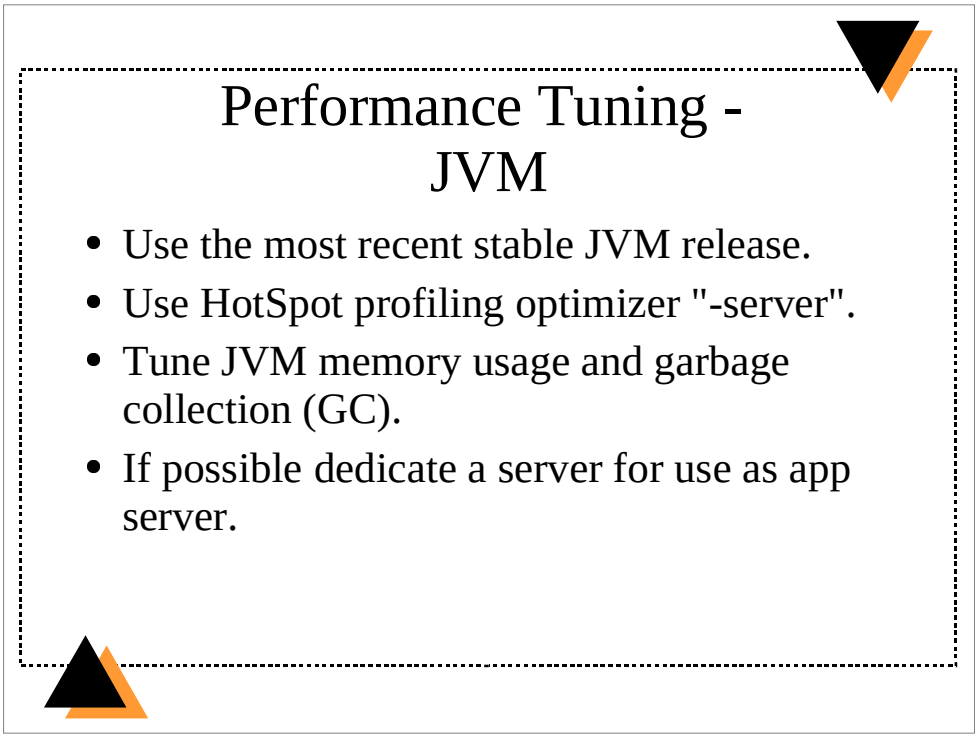
Introscope--Application Server Monitoring

Introscope can instrument any Java application and also supports JMX.

http://www.wilytech.com/

Example Shell Script

The example shell script in Appendix D can be used to detect major Tomcat problems. It forces the JVM to do a Thread Stacktrace dump and send e-mail notifications.

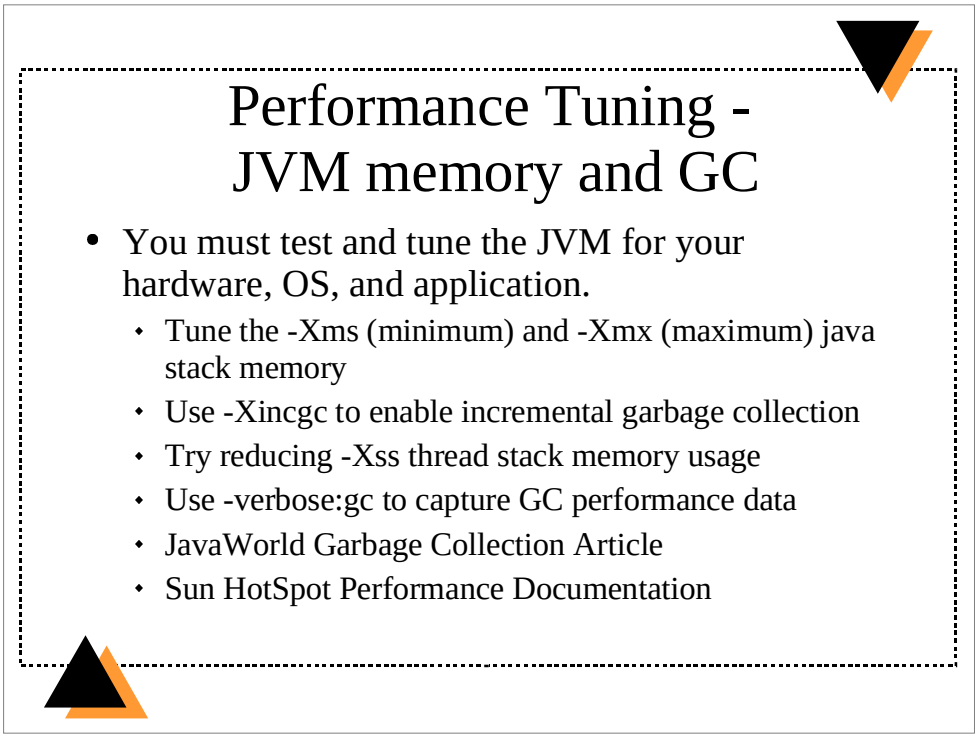


Notes

Sun Java 1.3 and later releases include the HotSpot profiling optimizer customized for long running server applications.

Sun Java 1.4 improves performance for servlet containers by approximately 35%.

Tomcat will freeze processing of all requests while the JVM is performing GC. On a poorly tuned JVM this can last 10's of seconds. Most GC's should take < 1 second and never exceed 10 seconds.



Notes

Setting the minimum and maximum java stack memory to the same value can improve GC performance.

Make sure the java process always keeps the memory it uses resident in physical memory and not swapped out to virtual memory.

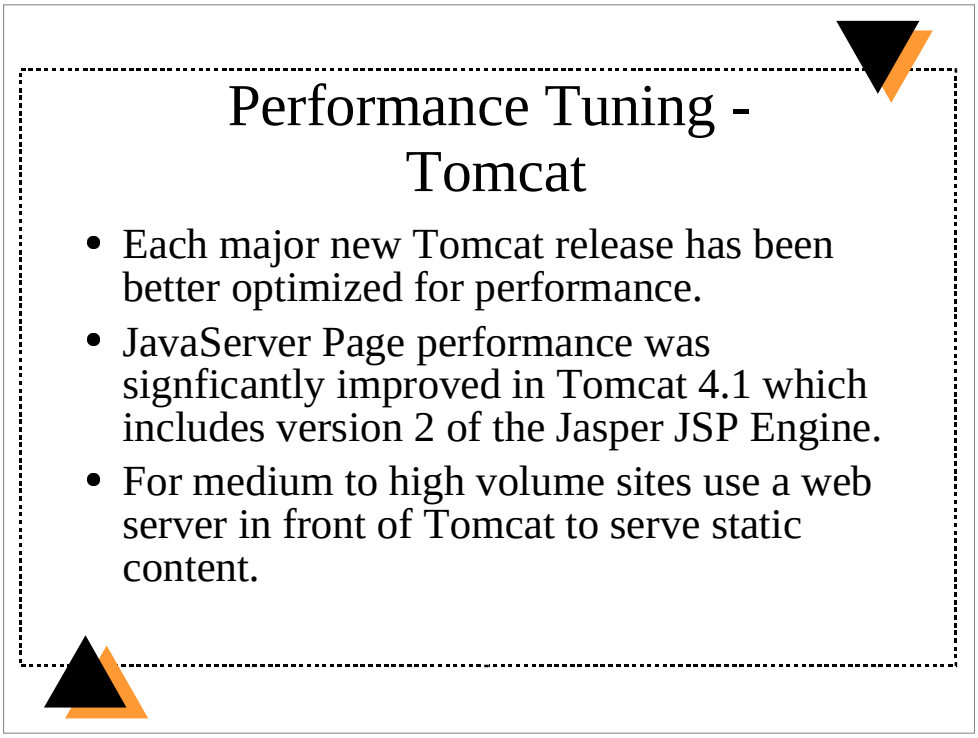
JVM garbage collection performance can degrade significantly if the JVM stack gets swapped out to disk as virtual memory.

JavaWorld Garbage Collection Article

http://www.javaworld.com/javaworld/jw-01-2002/jw-0111-hotspotgc.html

Sun HotSpot Performance Documenation

http://java.sun.com/docs/hotspot/index.html



Notes

Jasper 2

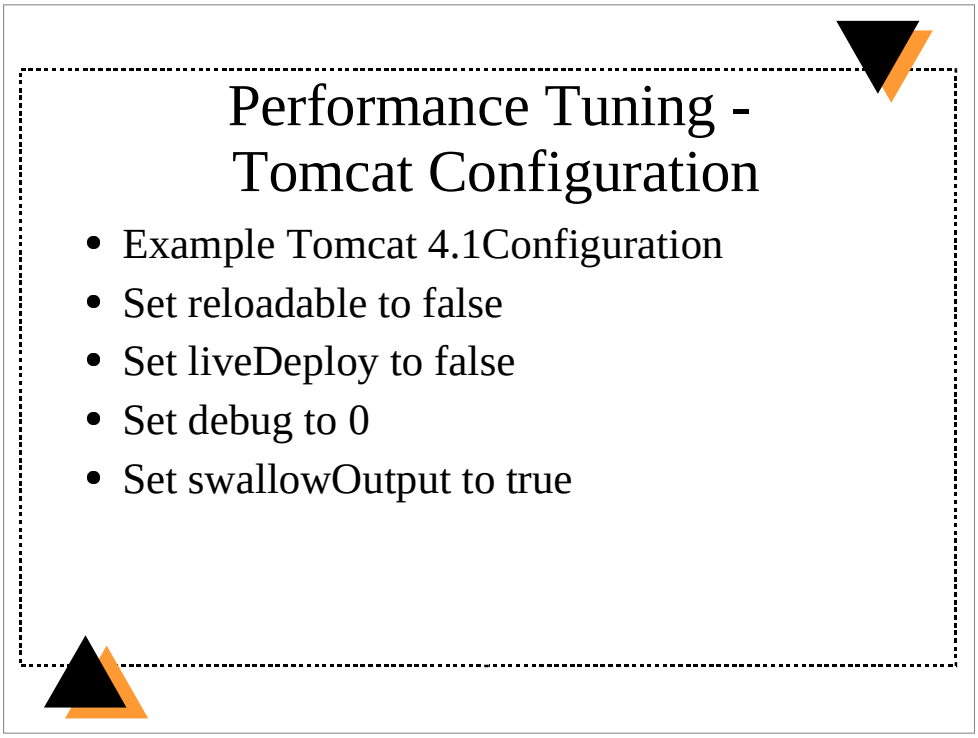
Can perform JSP page recompile in the background. Implements JSP custom tag pooling.

Web server

Let the web server do what it does best, serve static content. And let Tomcat do what it does best, generate dynamic content.

Apache 2.0

Apache mod_jk has better performance characteristics when you use Apache 2.0 with the threaded worker MPM.



Notes

See Appendix A -- Example Tomcat 4.1 Configuration

reloadable false

When reloadable is true Tomcat tries to detect web application class changes and automatically reload any classes which change. Setting this to false removes a lot of unnecessary overhead in production.

liveDeploy false

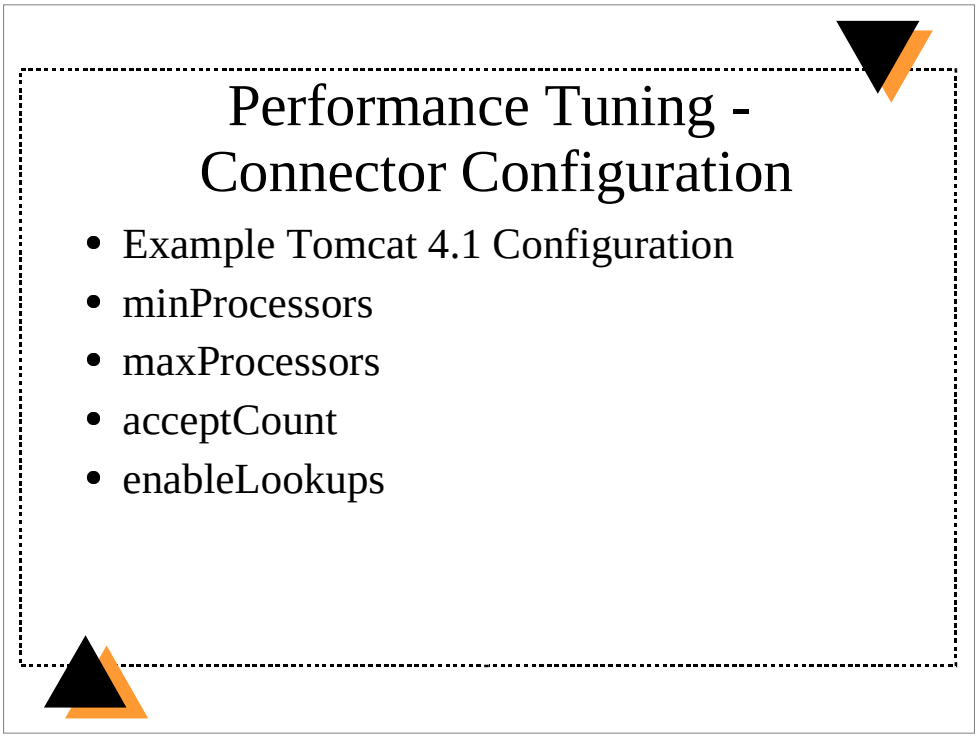
liveDeploy controls whether your webapps directory is periodically checked for new web application directories and war files. This is done using a background thread. Set this to false and use the manager application or ant deploy tasks instead.

debug 0

Disable all debug output, unnecessary logging just adds overhead.

swallowOutput true

This makes sure all output to stdout or stderr for a web application gets directed to the web application log rather than the console or catalina.out. This makes it easier to troubleshoot web application problems.



Notes

See Appendix A -- Example Tomcat 4.1 Configuration

minProcessors

Set to number of processors required for normal request volume.

maxProcessors

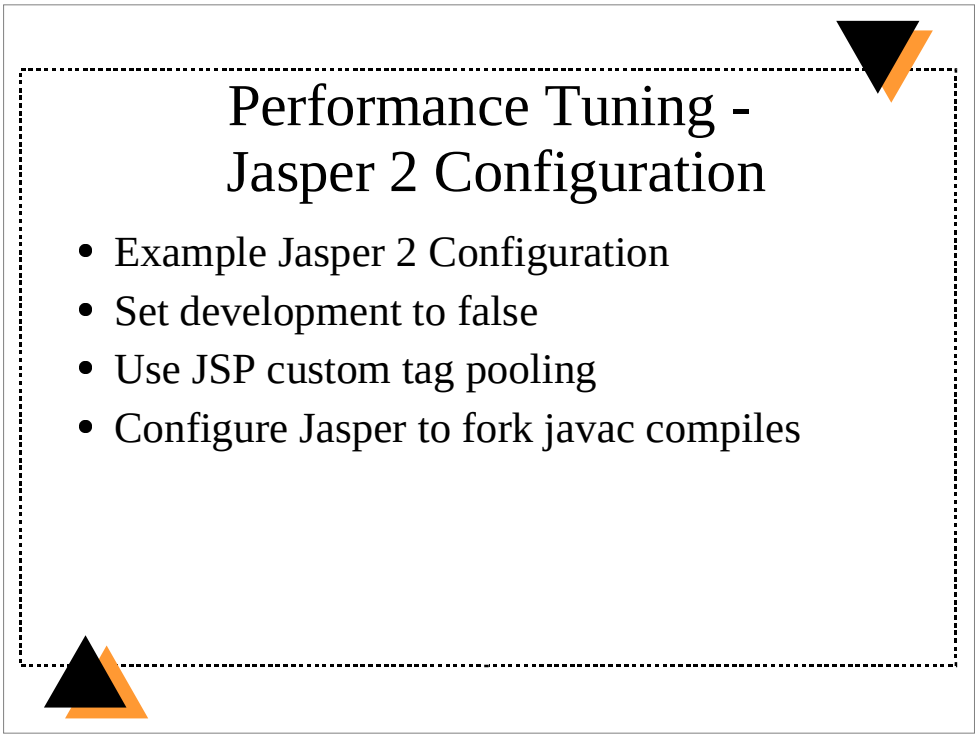
Set to twice the max number of concurrent requests expected.

acceptCount

Don't set too high, this sets the number of pending requests awaiting processing. In my opinion it is better to deny a few requests than overload Tomcat and cause problems for all requests.

enableLookups

DNS lookups can add significant delays.



Notes

See Appendix B -- Example Jasper 2 Configuration

development false

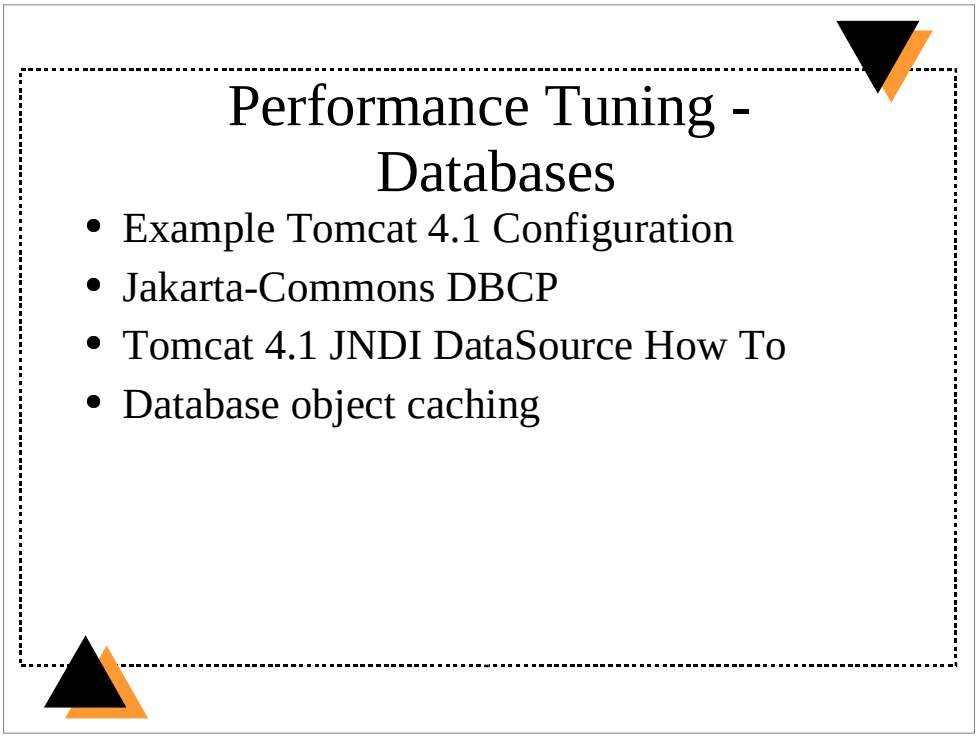
Disables JSP page out of date checks on each request and enables JSP background recompiles. development is set to true by default.

JSP custom tag pooling

Object pooling of classes which implement custom tags significantly improves performance of JSP pages which use custom tag libraries. JSP custom tag pooling is enabled by default.

Use fork for JSP page compiles

The JVM compiler javac has known memory leaks. Eliminates JVM memory usage and GC overhead of javac by configuring Jasper to fork javac when compiling JSP pages.



Notes

See Appendix A -- Example Tomcat 4.1 Configuration

Jakarta-Commons DBCP

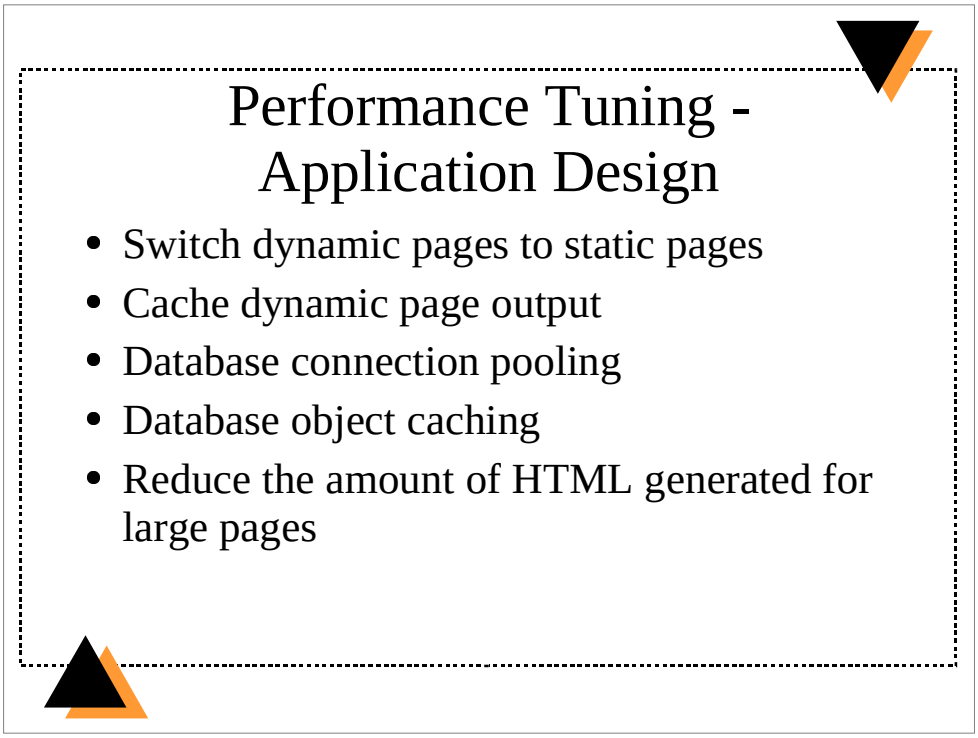
http://jakarta.apache.org/commons/dbcp.html

Tomcat 4.1 JNDI DataSource How To

http://jakarta.apache.org/tomcat/tomcat-4.1-doc/jndi-datasource-examples-howto.html

Database object caching

- Using middleware to persist and cache objects from your database can significantly improve performance.
- Fewer delays due to db query latency.
- Less thrashing of the JVM for creation and subsequent GC of objects created for result sets.
- Castor JDO http://castor.exolba.org/
- Torque http://db.apache.org/torque/
- Object Relational Bridge http://db.apache.org/



Notes

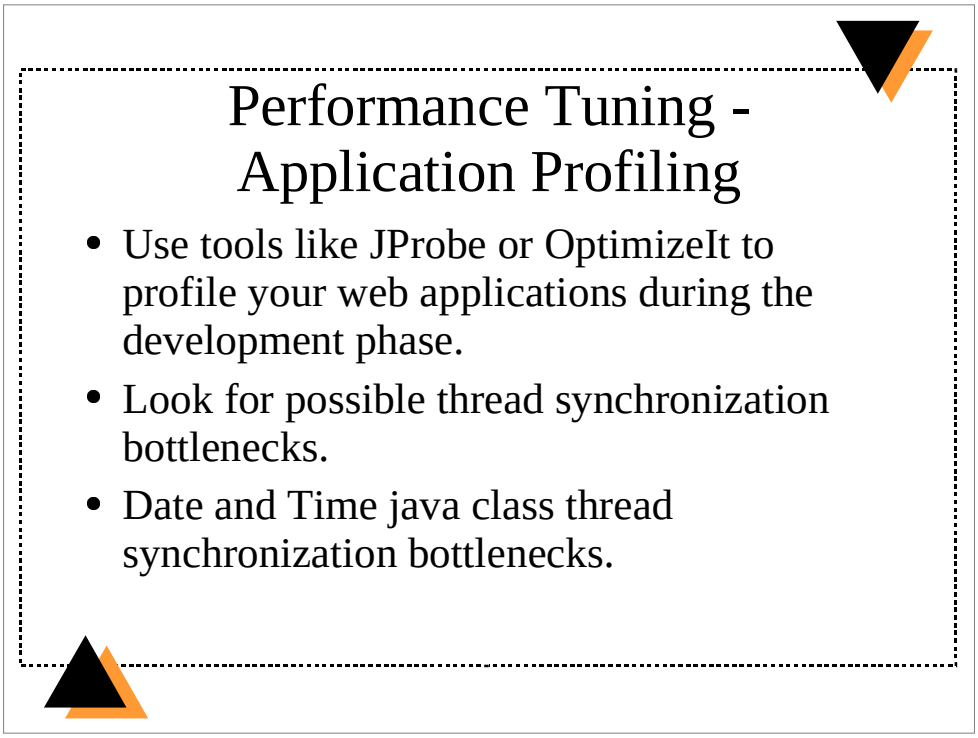
Switch dynamic pages to static pages

If the data used to generate a dynamic page rarely changes, modify it to a static page which you regenerate periodically.

Cache dynamic page output

If each request for a page doesn't generate unique output but changes too frequently to make it static, consider temporary caching of the generated output.

- Servlet 2.3 Cache Filter
  http://www.servletsuite.com/servlets/cacheflt.htm
- Jakarta-Taglibs Cache JSP tag library
  http://jakarta.apache.org/taglibs/doc/cache-doc/intro.html
- OSCache Servlet Filter
  http://www.opensymphony.com/oscache/



Notes

JProbe

http://www.sitraka.com/software/jprobe/

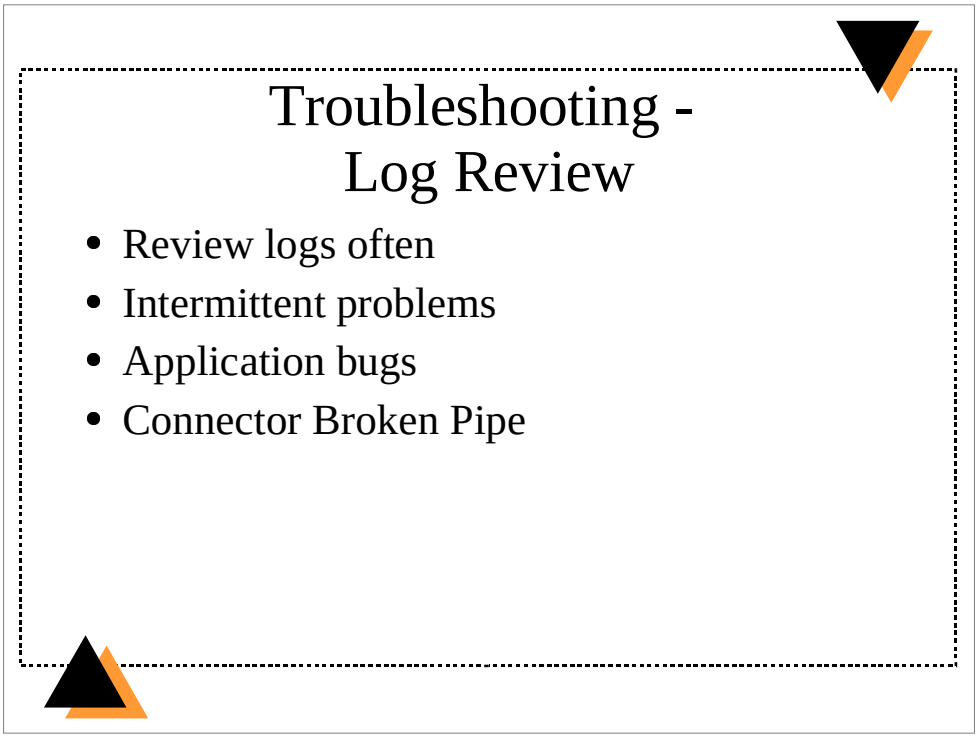
OptimizeIt

http://www.borland.com/optimizeit/

Date/Time synchronization bottleneck

The following java classes have a number of thread synchronization bottlenecks:

- java.sql.Date
- java.sql.Time
- java.sql.Timestamp
- java.text.SimpleDateFormat
- java.util.Calendar
- java.util.Date



Notes

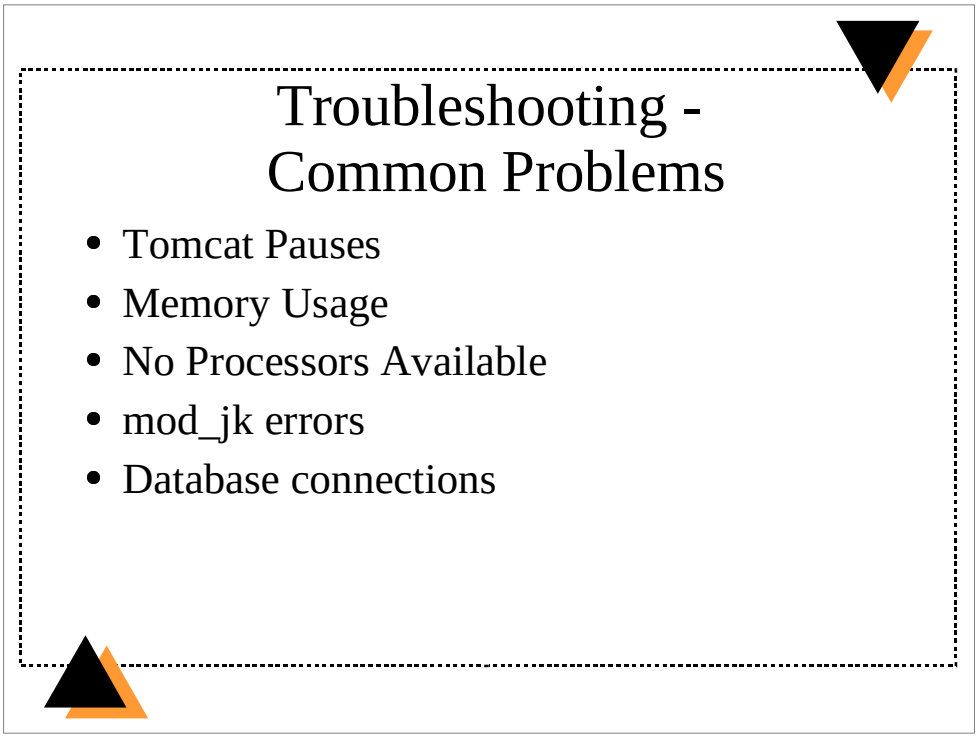
Review logs often, daily if possible.

Can help identify intermittent problems.

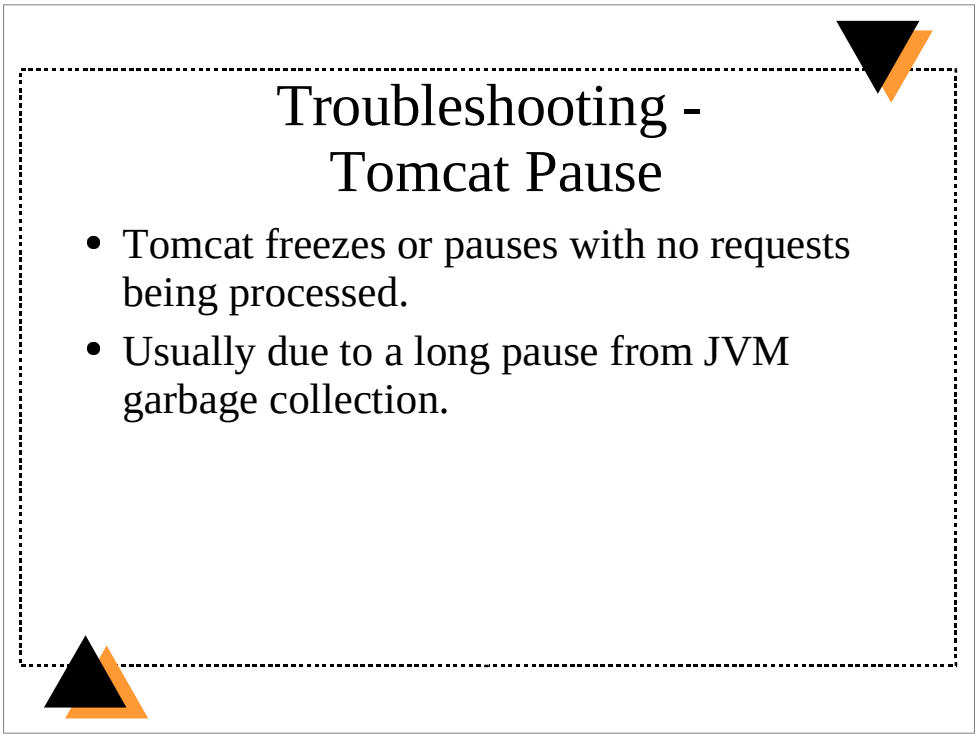
Application bugs can be found and fixed.

Connector Broken Pipe

- For HTTP Connector indicates that the remote client aborted the request.
- For a web server JK Connector indicates that the web server process or thread was terminated.
- These are normal and rarely due to a problem with Tomcat.


Troubleshooting -
Common Problems
Tomcat Pauses
Memory Usage
No Processors Available
mod_jk errors
Database connections

Notes

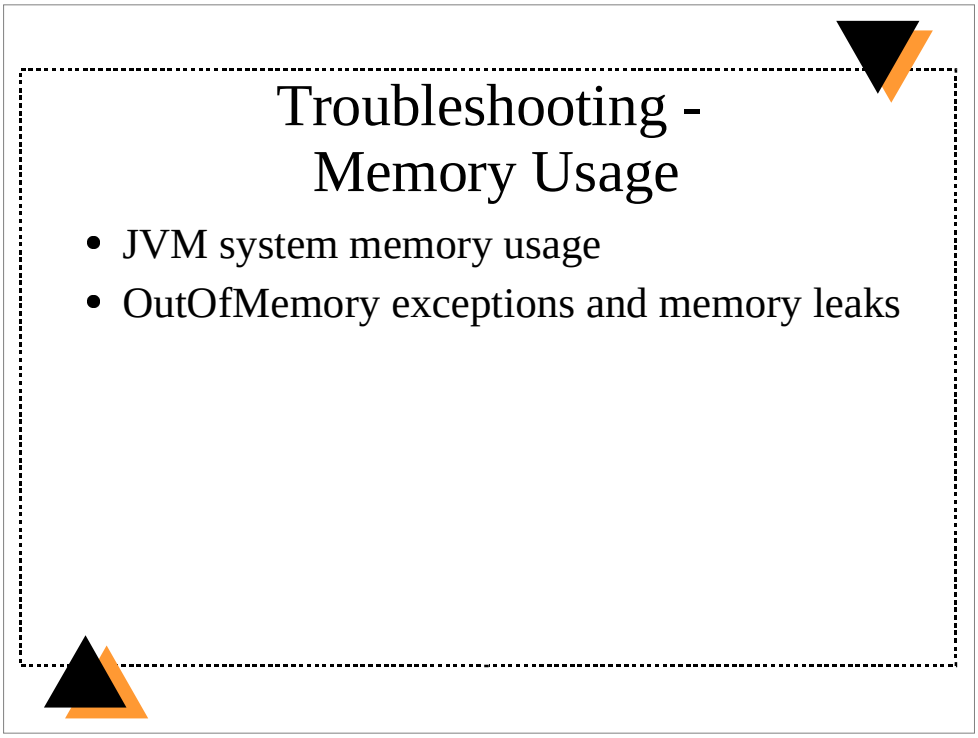


Notes

Use java –verbose:gc startup argument to collect GC data.

A long pause can cause a cascading effect and high load once Tomcat starts handling requests again.

Don't set the Connector acceptCount to high.



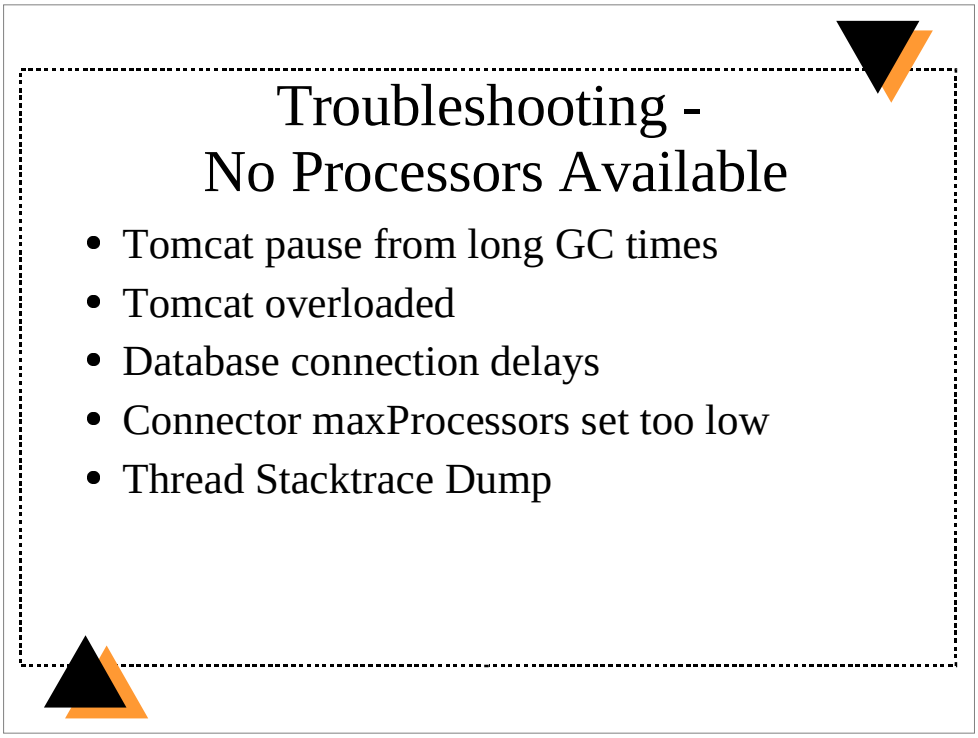
Notes

JVM system memory usage

- •JVM memory usage will increase over time to your -Xmx maximum memory config.
- •The JVM will never release that memory back to the OS.
- •The JVM manages its memory usage internally.
- •Use the java -verbose:gc option to collect data on JVM memory usage.

OutOfMemory exceptions and memory leaks

- •These are almost always caused by your application code.
- •Try increasing -Xmx if you have available memory on the system.
- •Stop and restart Tomcat
- •Profile your web application to determine where and why it is using excessive amounts of memory or leaking memory.



Notes

Tomcat overloaded

Review request volume, Tomcat CPU usage, request latency, and GC data.

Database connection delays

If there are database connection problems each request which uses the database could pause for as long as your db connection timeout. This puts processing for requests which use the db into a long wait. More and more Processors get used because too many requests are waiting for the db.

Connector maxProcessors set to low

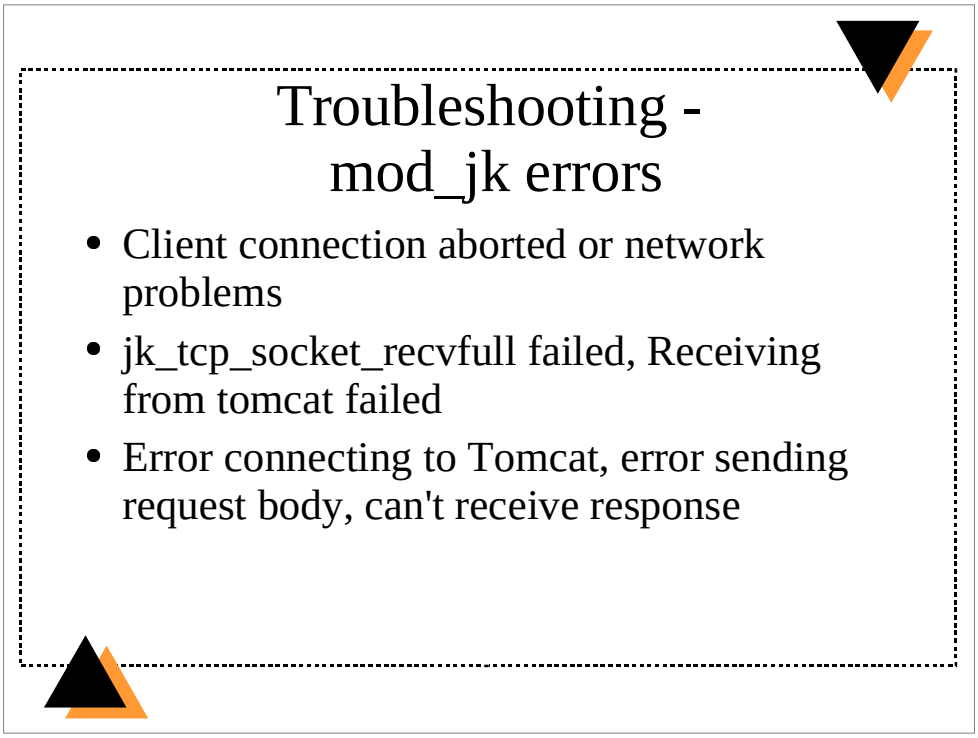
If there is no other problem causing the Connector to run out of Processors, your maxProcessor setting may be to low. I recommend setting it to twice the number of concurrent requests you expect Tomcat to process.

Apache mod_jk does a one-to-one mapping of httpd processes to Tomcat processors. If you have many web sites virtually hosted but not all of them use Tomcat, consider running a separate instance of Apache for just those sites that use Tomcat.

Thread Stacktrace Dump

If Tomcat is having problems, signal the JVM to do a Thread Stacktrace Dump. Reviewing the Stacktrace for each thread will often help you identify the source of the problem. Look for thread synchronization bottlenecks.

http://developer.java.sun.com/developer/technicalArticles/Programming/Stacktrace/



Notes

ERROR: Client connection aborted or network problems

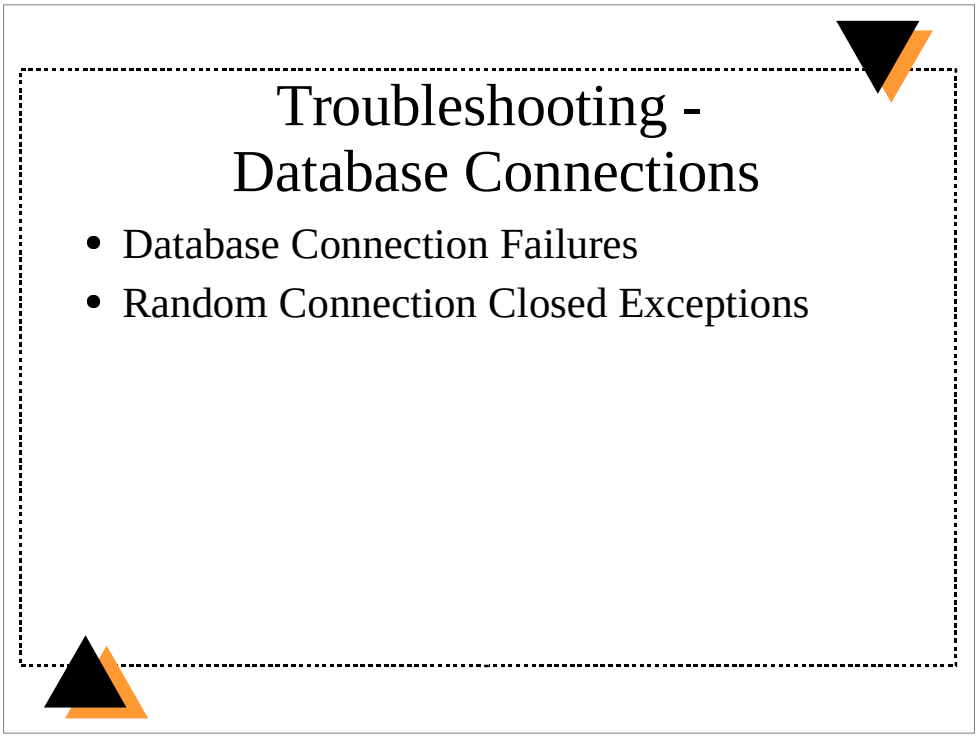
The remote browser client aborted the HTTP request. It is normal to see this error once in a while. This error occurs more frequently when request latency has increased due to Tomcat being overloaded.

Error - jk_tcp_socket_recvfull failed, Receiving from tomcat failed

All of Tomcats AjpProcessor's are in use and Tomcat rejected the connection. Tomcat is overloaded or the Connector maxProcessors needs to be increased.

Error connecting to the Tomcat process, error sending request, can't receive body

Either Tomcat isn't running, Tomcat and mod_jk are configured to use different ports, or a firewall is preventing the connection.



Notes

Database connection failures

•Long JVM GC times greater than db connection timeout can cause intermittent db connection failures.

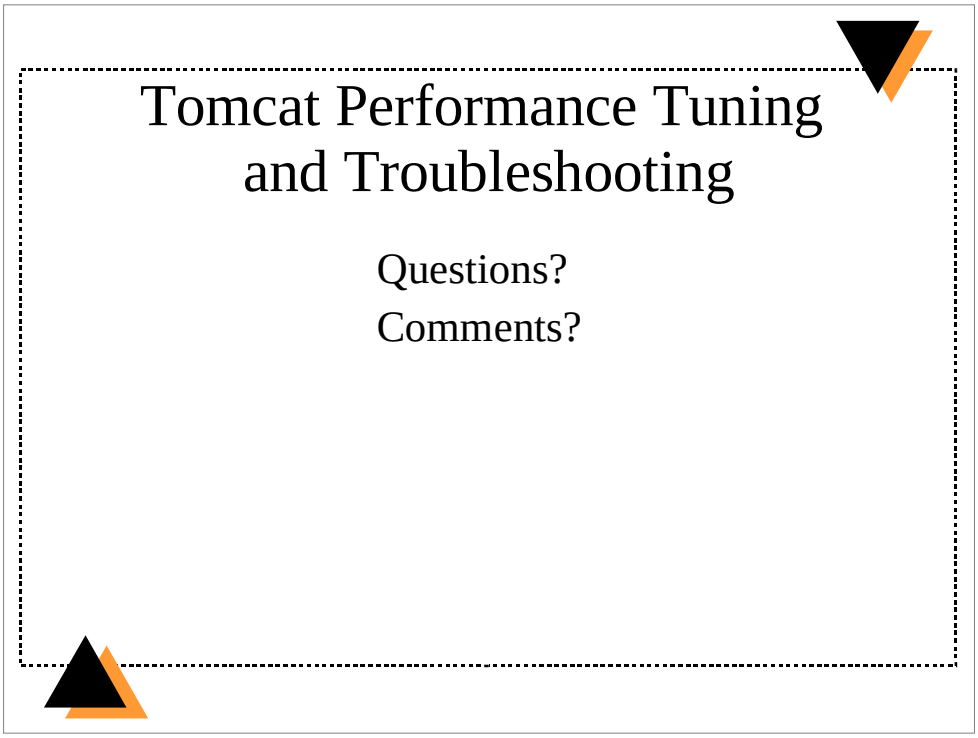
•Database connection pool out of connections due to abandoned db pooled connections which are never recycled.

•Spike in traffic to site using up all available db connections. This can either be in your db connection pool configuration or in your database server configuration.

Random Connection Closed Exceptions

•Usually caused by a bug in the web application usage of the db.

•See Appendix C -- Random Connection Closed Exceptions


Tomcat Performance Tuning
and Troubleshooting
Questions?
Comments?

Notes

This presenation has ended.
Click anywhere to return.

_

